

Juan José Agudelo Muñoz
Emanuel Henao Echeverry
Juan Pablo Montoya Rivera

Proyecto Final: Laberinto Inteligente

PixelAndes Games es una empresa de desarrollo de videojuegos educativos que busca crear experiencias para el aprendizaje de forma interactiva para jóvenes. Dentro de su línea de proyectos, el equipo estaba trabajando en un nuevo videojuego de laberintos: un desafío súper entretenido donde el que juega debe encontrar la salida navegando por ciertos caminos. Este debe evitar paredes y en ocasiones en algunas ocasionar se irá por caminos que pueden no tener salida. Sin embargo, antes de invertir en ello, se necesitaba un prototipo funcional que demostrara que era posible resolver laberintos de forma automática.

Este proyecto es parte del trabajo final de la materia Introducción a Ciencias de la Computación. El objetivo de PixelAndes se convierte en el contexto de nuestro proyecto. En este vamos a realizar un código en Colab que reciba un laberinto de una matriz y encuentre un camino libre desde la entrada hasta la salida usando backtracking, además tiene que marcar el recorrido que da solución al laberinto y también tiene que analizar y detectar si alguna matriz no tiene solución.

Descripción del problema

El problema consiste en encontrar un camino desde una celda de entrada S hasta una celda de salida G dentro de una matriz de celdas. Cada celda puede ser un camino libre en el que pondremos, en ese caso, un ".", también puede ser un obstáculo o pared el cual será un "#", o bien, la entrada o la salida. El agente que recorre el algoritmo puede moverse en cuatro direcciones las cuales son: arriba, abajo, izquierda y derecha. No puede pasar por encima de las paredes, salirse del límite de la matriz, ni pasar dos veces por la misma celda dentro del mismo recorrido.

La dificultad del problema radica en que hay algunos caminos que podrían parecer prometedores al principio, pero terminan bloqueados. El algoritmo debe poder explorar varias posibilidades y recordar por dónde ya pasó y retroceder cuando detecta que la ruya no sirve y no lleva a ningún lado. Por otro lado, también debería de reconocer en que momento una matriz no tiene un camino libre que lleve a la salida G y así poder devolver que la matriz no tiene solución posible.

Modelamiento con Backtracking

El algoritmo de backtracking está estructurado por la función `backtracking(posición_actual)`, que se llama de forma recursiva desde la función principal `resolver_laberinto()`. A continuación, describiremos como cada componente del backtracking se interpreta en el algoritmo construido:

Solución parcial: Es la lista camino, que contiene todas las posiciones visitadas hasta el momento en el recorrido actual. Se inicia con la posición de entrada y crece cada vez que se avanza a una nueva celda.

Candidatos: Son las cuatro celdas vecinas de la posición actual, generadas por la función obtener_vecinos(posición). En esta función se calcula las coordenadas de las celdas de arriba, abajo, izquierda y derecha.

Restricciones: La función es_válido(laberinto, posición, visitados) verifica tres condiciones antes de aceptar un candidato: que la celda esté dentro de los límites de la matriz (función esta_dentro), que no sea una pared y que no haya sido visitada ya en el camino actual.

Solución completa: Se llega a ella cuando posicion_actual == salida, es decir, cuando la posición actual coincide con la coordenada de la celda G. En ese momento la función retorna True y el camino acumulado en la lista camino es la solución.

Retroceso: Si ningún vecino válido lleva a la solución, la función elimina la última posición agregada con camino.pop(), la quita del conjunto de visitados con visitados.remove(vecino), y retorna False para que el nivel anterior intente otra alternativa. Esto es lo que significa el retroceso en el código, deshacer la última decisión.

Poda: El conjunto visitados tiene el objetivo que una poda tendría: antes de explorar un vecino se verifica que no esté ya en ese conjunto. Esto evita tanto ciclo y tanta repetidera; descarta ramas completas sin necesidad de explorarlas hasta el fondo.

Ejemplo pequeño:

Para este ejemplo tomemos un laberinto sencillo de 3x3 para poder ilustrar el proceso paso a paso:

	C0	C1	C2
F0	S	.	#
F1	.	.	.
F2	#	.	G

El algoritmo comienza en (0,0) y evalúa sus vecinos. Hacia arriba: fuera de la matriz. Hacia la izquierda: fuera de la matriz. Hacia abajo: (1,0), celda libre → se avanza. Desde (1,0) se intenta ir a (0,0) (ya visitado, descartado) y hacia (1,1) (libre → se avanza). Desde (1,1), se pasa a (2,1) y luego (2,2) = G. Solución encontrada: (0,0) → (1,0) → (1,1) → (2,1) → (2,2).

Si hubiera una pared bloqueando (1,1), el algoritmo retrocedería desde (1,0) hasta (0,0) y probaría ir hacia (0,1). Si ese camino tampoco lleva a G, la función devuelve False y se reporta que no hay solución.

Qué encontramos

Se probaron tres algoritmos de 6x6. El primero tenía solución, el camino solución fue: $(0,0) \rightarrow (0,1) \rightarrow (0,2) \rightarrow (1,2) \rightarrow (2,2) \rightarrow (2,3) \rightarrow (2,4) \rightarrow (2,5) \rightarrow (3,5) \rightarrow (4,5) \rightarrow (5,5)$, marcado con “*” en la matriz. El algoritmo también mostró la cantidad de llamadas recursivas, ramas descartadas y retrocesos. Así pues, este exploro y descarto antes de llegar a la solución completa.

El segundo laberinto tenía una pared vertical completa en la columna 2, separando la entrada de la salida sin ningún hueco. El algoritmo evaluó las posibilidades del lado izquierdo y devolvió False, así comunicándonos que no había forma de llegar a G.

El tercer laberinto tenía varias rutas posibles. El algoritmo encontró la primera ruta válida según el orden de validación de vecinos (arriba, abajo, izquierda, derecha) y se detuvo sin necesidad de encontrar todas. En este se vio números más grandes de llamadas recursivas en comparación con el primer caso, así que el código examinó muchos más caminos.

Decisiones del grupo

De todos los proyectos que había para escoger, decidimos trabajar con el laberinto debido a que nos pareció el más divertido de trabajar, retador y además de eso, porque es un proyecto muy popular en la programación y uno de los mejores ejemplos para explicar lo que significa trabajar con backtracking. En las clases, el profesor también afirmó que la solución de laberintos es una buena forma visual de entender esta técnica de búsqueda.

La representación que elegimos fue una matriz, donde cada posición corresponde a una celda del laberinto. Algunas celdas representan caminos libres, otras paredes, y también se definen una posición de inicio y una de salida.

Las primeras restricciones que implementamos fueron: verificar que el movimiento no saliera de los límites de la matriz y que no pasara por una pared. Después agregamos la restricción de no volver a pasar por una celda que ya había sido visitada durante el recorrido, ya que esto podía hacer que el algoritmo entrara en ciclos y nunca encontrara la solución.

La poda que utilizamos consiste en descartar las celdas que ya fueron visitadas. De esta forma evitamos recorrer una y otra vez los mismos lugares. Con esta se puede reducir el número de caminos que el algoritmo necesita revisar.

Si hubiéramos tenido más tiempo, hubiera sido interesante llevar el proyecto a un videojuego de verdad. En lugar de mostrar el laberinto como una matriz de texto, mostrarlo gráficamente y animar el recorrido del algoritmo para observar en tiempo real cómo examina los caminos libres, queda atrapado, retrocede y finalmente encuentra la salida.

Para finalizar este informe, hay que recalcar que fue un proyecto muy chévere, es una muy buena forma de interiorizar el concepto de backtracking en vez de verlo de otra manera más teórica, aunque igual la vimos en clase. Así que pudimos observar mejor los pasos en los que se resuelve un problema con esta técnica y llegar a la solución completa.

En general, el programa cumple con los objetivos planteados: encuentra un camino cuando existe, identifica cuándo no hay solución, marca el recorrido encontrado y registra información sobre la ejecución del algoritmo. Para desarrollarlo nos inspiramos en la lógica utilizada en el proyecto ROBERT5, adaptándola al contexto del laberinto y simplificando varias partes para que el código fuera más fácil de entender y explicar.